

Szakdolgozat készítése – Nánási Bence

Tesztkörnyezet létrehozása

A projektben való munkám elkezdéséhez először meg kellett vizsgálnom a rendelkezésre álló „erlangbert” repository-t. A célom az volt, hogy létrehozzak egy lokális környezetet, amelyben beüzemelem az elérhető megoldásokat és elkezdjem tesztelni a működésüket.

Első lépésként leklónoztam a repository-t, majd felállítottam a virtuális python környezetet a megfelelő függőségek telepítésével.

```
git clone https://github.com/hsnlab/erlangbert.git
cd erlangbert
python -m venv .venv
.\venv\Scripts\Activate.ps1

pip install -r requirements.txt
```

A GraphCodeBERT alapú modellek a forráskód strukturális információit is felhasználják. Ehhez szükségem volt az Erlang nyelv tree-sitter parserére. A third_party/tree-sitter-erlang könyvtárban található forrásból le kellett fordítanom a dinamikus könyvtárat, mivel Windows környezetben vagyok, ezért .dll formátumban. Létrehoztam egy egyszerű build scriptet, majd futtattam:

```
from tree_sitter import Language
Language.build_library(
    r"parsers\erlang.dll",
    [r"third_party\tree-sitter-erlang"]
)
print("Built parsers/erlang.dll")
```

A következő lépés az Erlang-kódbázisok letöltése volt a GitHubról. Ehhez a beépített repository discovery pipeline-t használtam, amely automatikusan kiválaszt néhány népszerű, jó minőségű Erlang projektet. A GitHub API használatához megadtam a saját GitHub tokenemet, majd elindítottam az adat-előkészítő lépést:

```
python .\main.py prepare `
--github-token $env:GITHUB_TOKEN `
--max-repos 5 `
--min-stars 20 `
--max-parser-workers 4 `
--graphcodebert-output output\graphcodebert_data `
--force-refresh `
```

Ez a parancs automatikusan felfedezett 5 népszerű Erlang projektet (pl. cowboy, ejabberd, vernemq, couchdb, theBeamBook), klónozta ezeket a cloned_repos könyvtárba, majd a beépített ErlangParser segítségével kinyerte a függvényeket és azok DFG-adatait. A feldolgozás után elkészültek az tanítási adatfájlok:

```
output\graphcodebert_data\train.jsonl
output\graphcodebert_data\valid.jsonl
output\graphcodebert_data\test.jsonl
output\graphcodebert_data\full.jsonl
```

Ezek a JSONL fájlok tartalmazzák a tokenizált forráskód-szegmenseket, a függvényneveket, a kapcsolódó gráfokat és metaadatokat.

Ezután megnéztem, milyen opciókat támogat a main.py a tanításhoz és ez alapján megírtam a tanításhoz a parancsot, amelyekben beállítottam a számomra fontos paramétereket állító kapcsolók értékeit.

A cél egy gyors, erőforrás-takarékos finomhangolás volt, ezért a LoRA opciót használtam, ami jelentősen csökkenti a számításigényt. A tréninget a teljes korpuszon indítottam:

```
python .\main.py train `
--train-file output\graphcodebert_data\train.jsonl `
--val-file  output\graphcodebert_data\valid.jsonl `
--use-lora `
--model-output-dir models\erlangbert_lora_full `
--log-file logs\train_lora_full.log
```

Ez a lépés betöltötte az alap GraphCodeBERT modellt, inicializálta a LoRA adaptereket az új paraméterekhez, majd elindította a tanítást a train.jsonl adatokon, validációs metrikákat mentett minden eval_steps ciklus után. A futási idő körülbelül 3 órára becsülhető volt a teljes adathalmazon.